



Software Requirements Specification (SRS)

Version 1.0

Crop Health Monitoring and Recommendation System for Farmers

Submitted by: Team AgriVision

1. Introduction

1.1 Purpose

This document provides a comprehensive specification for a web-based Crop Health Monitoring and Recommendation System. The system enables farmers to upload crop images, which are analyzed by a machine learning model to detect potential diseases and provide actionable recommendations. The SRS serves as a reference for both developers and stakeholders.

1.2 Scope

The system is intended to assist farmers in diagnosing crop health issues through an accessible web application. It will support:

- User authentication (sign up, login, reset password)
- Image upload functionality
- Automated disease classification using ML (CNN + ResNet/DenseNet)
- Delivery of recommendations based on disease detection
- Notifications and report history
- Secure data handling and user management

1.3 Glossary

Term	Definition
CNN	Convolutional Neural Network for image classification
ResNet50	A deep residual learning model for image generalization
ML Model	Trained model to predict disease based on crop images
API	Application Programming Interface
Softmax	Final activation layer in a classifier model

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- TensorFlow Documentation
- MongoDB Atlas Documentation

1.5 Overview of Document

This document includes a high-level system description, functional and non-functional requirements, system architecture, and use case descriptions.

2. Overall Description

2.1 System Environment

- **Frontend:** Built in React.js; supports mobile and desktop; interfaces with backend via REST APIs.
- **Backend:** Node.js + Express; manages APIs, ML inference calls, DB operations.
- **ML Engine:** Python-based TensorFlow model invoked from backend using subprocess.
- **Database:** MongoDB for users, uploads, and results.

2.2 Functional Requirements

2.2.1 User Authentication

- Users can register via email and password.
- Users can log in and reset password using verified email or mobile number.

2.2.2 Profile Management

- Users can view and edit profile details.
- Option to log out securely.

2.2.3 Image Upload and Disease Analysis

- Upload crop images (.jpg/.png)
- Images are preprocessed (resized, normalized)
- Passed through ResNet50/DenseNet for generalization
- CNN classifies diseases
- Softmax layer outputs probabilities
- Output $< 0.7 \rightarrow$ Healthy
- Output $\geq 0.7 \rightarrow$ Diseased
- Recommendations are generated accordingly

2.2.4 Notifications and Reports

- Notification system shows results of latest analysis
- View history of past uploads and recommendations

2.3 User Characteristics

- Farmers with basic smartphone literacy
- Admin panel access for developers/support staff

2.4 Non-Functional Requirements

- System should respond within 5 seconds to inference requests
 - Secure user authentication using JWT
 - Data encryption for images and personal information
 - Availability: 99% uptime
 - Scalability: Handle 100 concurrent users
-

3. Requirements Specification

3.1 External Interfaces

- **User Device:** Browser or mobile device with camera
- **ML Model:** Invoked via Python subprocess from Node backend
- **Database:** MongoDB Atlas cloud-based connection

3.2 Detailed Functional Requirements

Use Case 1: Sign Up/Login

Trigger: User opens the app **Description:** Provides secure access to the system via email/password

Use Case 2: Upload Image

Trigger: User selects an image for diagnosis **Description:** Uploads image, triggers ML processing, shows result

Use Case 3: View Notifications

Trigger: System detects disease **Description:** Notifies user with diagnosis result and recommendation

Use Case 4: Edit Profile

Trigger: User accesses profile section **Description:** Allows updates to name, email, password

Use Case 5: View History

Trigger: User opens report section **Description:** Shows all past uploads, diagnoses, and recommendations

3.3 Logical Structure of the Data

- **Users**

- user_id

- name

- email

- password_hash

- **Images**

- image_id

- user_id

- file_path

- timestamp

- **Results**

- image_id

- disease_class

- confidence

- recommendation

4. Security and Privacy

- JWT-based secure login

- HTTPS encrypted API calls

- Images and data stored with access control

5. Appendices

- Dataset Source: Indian crop disease datasets (e.g., PlantVillage, ICAR)

- Hardware Requirements: Server with GPU recommended for fast inference

- Model: Trained TensorFlow model (.h5)

End of Document

thank you